

2 - Control PWM

Para implementar correctamente nuestro péndulo invertido debemos tener una forma de controlar la velocidad de los motores desde nuestro microcontrolador.

La forma más común de hacer esto es mediante PWM (Modulación de ancho de pulso). El PWM es una técnica que consiste en generar una señal pulsante a frecuencia constante e ir variando el ciclo de trabajo..

Si conectamos un motor a un generador PWM, la velocidad del motor dependerá del ciclo de trabajo, es decir, que para un ciclo de trabajo fijo, el motor girará a velocidad constante. A frecuencias más altas, más preciso es nuestro control.

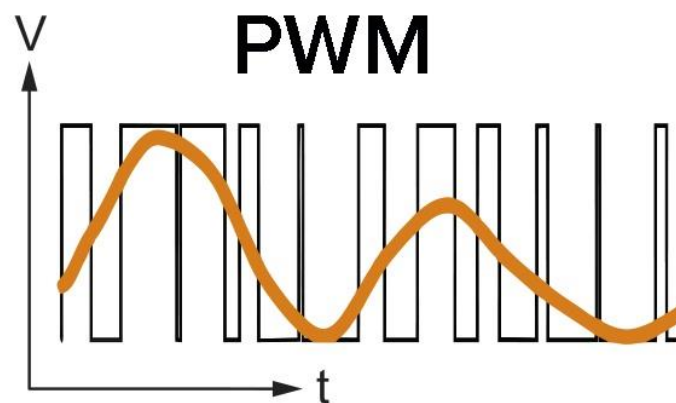
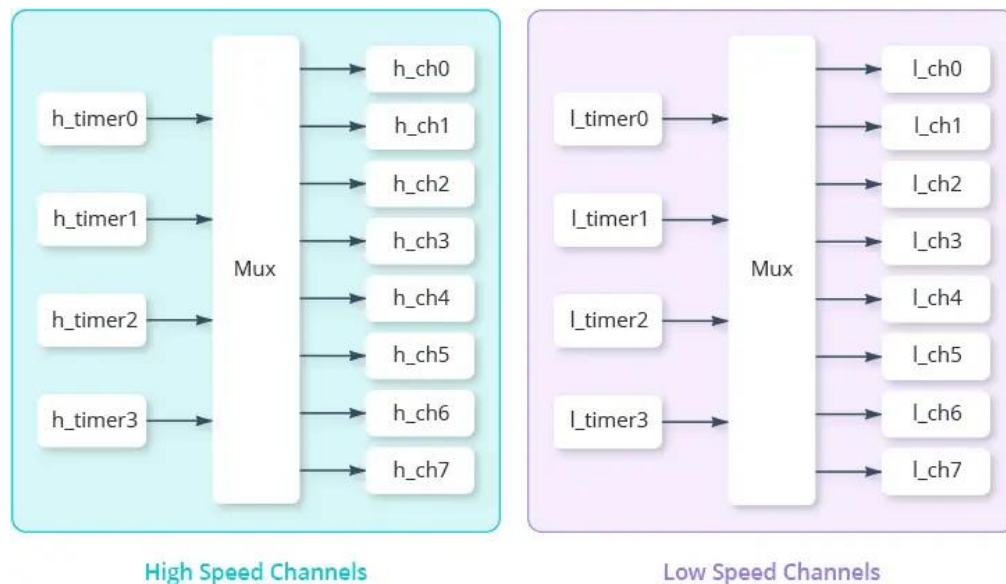


figura 2.1. La curva amarilla representa el valor promedio en cada momento de esta señal PWM

Entonces, lo que vamos a hacer es implementar un control PWM para el ESP32.

El código

El ESP32 tiene 16 canales internos de PWM, los primeros 8 de alta frecuencia y los otros de baja frecuencia.



Los canales de alta velocidad están implementados en el hardware, es decir que son independientes del procesador, mientras que los canales de baja frecuencia dependen del software para cambiar el ciclo de trabajo.

Como puede verse en la imagen anterior, cada 8 canales hay cuatro timers, por lo que cada temporizador es compartido por dos canales. Esto significa que los dos canales que compartan un mismo temporizador tendrán la misma frecuencia, pero su ciclo de trabajo se puede controlar de forma independiente. **En nuestro caso esto no importa, ya que la frecuencia de PWM que vamos a usar para los motores va a ser la misma para cada salida.**

Para generar una señal PWM, debemos “adjuntar” un canal de PWM a un pin GPIO. Es posible adjuntar el mismo canal a varios pines. El código es el siguiente:

```
#include <Arduino.h>

//Definimos lo pines de salida del motor que se controla con un puente H
uint8_t pin1 = 2;
uint8_t pin2 = 16;
uint8_t ch0 = 0;
uint8_t ch1 = 1;

void setup()
{
    int freq = 30000; //La frecuencia de la señal PWM. Usamos 30kHz ya que
    //está fuera de la frecuencia audible
    int res = 1023; //La resolución del PWM
    uint8_t res_bits = 10; //La misma resolución en bits

    //Elegimos los canales de PWM
    //Establece los pines como salida
    pinMode(pin1, OUTPUT);
    pinMode(pin2, OUTPUT);

    //Establece la frecuencia y la resolución del PWM en los canales 0 y 1
    ledcSetup(ch0, freq, res_bits);
    ledcSetup(ch1, freq, res_bits);

    //Adjunt los pines pin1 y pin2 a los canales ch0 y ch1, respectivamente
    ledcAttachPin(pin1, ch0);
    ledcAttachPin(pin2, ch1);

    //Pone el ciclo de trabajo en 0 para que los motores estén apagados
    ledcWrite(ch0, 0);
    ledcWrite(ch1, 0);
}
```

Luego, en el loop, llamando a la función `ledcWrite`, podemos cambiar el ciclo de trabajo de los distintos pines.

Bibliografía

- <https://lastminuteengineers.com/esp32-pwm-tutorial/#:~:text=PWM%20Channels,-Now%20let%20us&text=A%20channel%20represents%20a%20unique,and%208%20low-speed%20channels.>
- <https://randomnerdtutorials.com/esp32-pwm-arduino-ide/>